

Advanced API Development & Security

Filtering, searching, ordering, pagination, caching, data sanitization, authentication, Djoser, JWT, and rate throttling in DRF.

1. Filtering, Searching & Ordering in DRF

DRF provides built-in filter backends to query database records dynamically without writing custom SQL or manual ORM queries for every parameter.

DjangoFilterBackend

Exact field matching (e.g., `?category=1&price=10`).
Requires `django-filter`.

SearchFilter

Performs text searches on string fields (e.g., `?search=pizza`) using SQL `LIKE` queries.

OrderingFilter

Sorts results based on allowed fields (e.g., `?ordering=-price` for descending order).

```
# views.py
from rest_framework import generics
from rest_framework.filters import SearchFilter, OrderingFilter
from django_filters.rest_framework import DjangoFilterBackend
from .models import MenuItem
from .serializers import MenuItemSerializer

class MenuItemView(generics.ListCreateAPIView):
    queryset = MenuItem.objects.all()
    serializer_class = MenuItemSerializer
    filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
    filterset_fields = ['category', 'price']
    search_fields = ['title', 'category__title']
    ordering_fields = ['price', 'inventory']
```

2. Data Validation & Sanitization

Validation ensures input conforms to expected data types and business rules. **Sanitization** strips out potentially harmful content (like malicious HTML/JS tags) to prevent cross-site scripting (XSS) and code injection.

```
# Sanitize input HTML using bleach
import bleach
class MenuItemSerializer(serializers.ModelSerializer):
    def validate_title(self, value):
        # Clean HTML tags from string field
        return bleach.clean(value)
```

3. Pagination in DRF

Pagination limits response payload size, reducing database memory usage and improving load times.

Pagination Style	Query Parameter Example	Characteristics
PageNumberPagination	<code>/api/menu-items/?page=2</code>	Standard page-based navigation. Common for frontend tables.
LimitOffsetPagination	<code>/api/menu-items/?limit=10&offset=20</code>	Flexible offset querying; useful for SQL-like data fetches.

4. Caching for API Performance

Caching reduces database overhead by storing rendered API responses in memory (e.g., Redis or Memcached) for repeat requests.

```
from django.utils.decorators import method_decorator from django.views.decorators.cache
import cache_page class MenuItemsView(generics.ListAPIView): queryset = MenuItem.objects.all()
serializer_class = MenuItemSerializer # Cache view response for 2 minutes (120 seconds)
@method_decorator(cache_page(60 * 2)) def get(self, request, *args, **kwargs): return
super().get(request, *args, **kwargs)
```

5. DRF Token Authentication & User Roles

DRF provides a built-in token authentication system where clients exchange credentials for an HTTP Authorization token key.

Token Setup in settings.py

```
INSTALLED_APPS = [ ...
'rest_framework.authtoken', ] REST_FRAMEWORK = {
'DEFAULT_AUTHENTICATION_CLASSES': [
'rest_framework.authentication.TokenAuthentication',
], }
```

Authorization Header Format

Clients pass the issued token in HTTP headers subsequent requests:

```
Authorization: Token
9944b09199c62bcf9418ad846dd0e4bbdfc
```

6. User Roles & Group Permissions

Assign users to Django **Groups** (e.g., Manager, Crew, Customer) to restrict actions programmatically inside API views or permission classes.

```
from rest_framework.permissions import BasePermission class IsManager(BasePermission): def
has_permission(self, request, view): return request.user.groups.filter(name='Manager').exists()
class MenuItemDetailView(generics.RetrieveUpdateDestroyAPIView): queryset =
MenuItem.objects.all() serializer_class = MenuItemSerializer def get_permissions(self): if
self.request.method in ['POST', 'PUT', 'PATCH', 'DELETE']: return [IsManager()] return []
```

7. API Throttling (Rate Limiting)

Throttling controls the rate of requests clients can make to an API, protecting servers against DDoS attacks, brute-force attempts, and resource exhaustion.

Throttle Class	Target Audience	Settings Configuration Example
AnonRateThrottle	Unauthenticated users	'anon': '100/day'
UserRateThrottle	Authenticated users	'user': '1000/day'
ScopedRateThrottle	Specific custom endpoints	Set throttle_scope = 'uploads' on view

```
# settings.py REST_FRAMEWORK = { 'DEFAULT_THROTTLE_CLASSES': [  
'rest_framework.throttling.AnonRateThrottle', 'rest_framework.throttling.UserRateThrottle', ],  
'DEFAULT_THROTTLE_RATES': { 'anon': '10/minute', 'user': '100/minute', } }
```

8. Djoser & JWT (JSON Web Tokens)

Djoser provides ready-made endpoints for user registration, password resets, and account management. Paired with SimpleJWT, it provides secure stateless authentication using Access and Refresh tokens.

Token Lifetime Cycle

- Access Token:** Short-lived key (~5–15 mins) attached to HTTP Authorization: Bearer <token> .
- Refresh Token:** Long-lived key (~1–7 days) used to fetch new access tokens without re-entering credentials.

Key Djoser Endpoints

- /auth/users/ — User registration & listing
- /auth/users/me/ — Current profile management
- /auth/jwt/create/ — Obtain JWT access/refresh pair
- /auth/jwt/refresh/ — Refresh expired access token

9. Summary Checklist

API Production Readiness

- Implement searching, filtering, and ordering to give clients targeted data access.
- Sanitize user input with packages like bleach to guard against security vulnerabilities.
- Configure throttling rates to defend against abuse and excessive API calls.
- Enforce role-based access control (RBAC) through custom permission classes.